Supplementary Information

Same Prompt, Different Answer: Hidden Non-Determinism in LLM APIs Undermines Scientific Reproducibility

Lucas Rover, Hugo Valadares Siqueira, Eduardo Tadeu Bacalhau, Anibal Tavares de Azevedo & Yara de
UTFPR – Universidade Tecnológica Federal do Paraná

This document provides Supplementary Information for the main manuscript. Sections S1–S13 contain full prompt templates, input descriptions, protocol analyses, statistical details, experimental coverage data, the code, mathematics, and cross-domain task details (S11), the two-judge LLM-as-judge triangulation protocol (S12), and the per-field reproducibility analysis (S13), referenced in the main text.

Contents

S1 Full Prompt Templates

The exact prompt templates used for each of the four experimental tasks are reproduced below. In all templates, {`abstract`} is replaced at runtime with the full text of the target scientific abstract, and {`retrieved_passage`} is replaced with the domain-relevant context passage for RAG tasks.

S1.1 Task 1: Scientific Summarization

Listing 1: Prompt template for Task 1 (scientific summarization).

```
Summarize the following scientific abstract in exactly
3 sentences. Cover: (1) the main contribution,
(2) the methodology used, and (3) the key quantitative
result.

Abstract: {abstract}

Summary:
```

This is an open-ended generation task. The model must produce three sentences of plain text with no structured markup. The unconstrained output format permits substantial lexical variation across runs.

S1.2 Task 2: Structured Extraction

Listing 2: Prompt template for Task 2 (structured extraction).

```
Extract the following fields from the scientific abstract
below. Return JSON only, no explanation.

Fields: objective, method, key_result, model_or_system,
benchmark

Abstract: {abstract}

JSON:
```

This is a constrained generation task. The model must return a JSON object with exactly five fields. The structured output format reduces but does not eliminate lexical variation.

S1.3 Task 3: Multi-Turn Refinement (3 turns)

Listing 3: Prompt template for Task 3 (multi-turn refinement, 3 turns).

```
Turn 1:
Summarize the following scientific abstract in exactly
3 sentences. Cover: (1) the main contribution,
(2) the methodology used, and (3) the key quantitative
result.

Abstract: {abstract}

Summary:
```

```
Turn 2:
Now revise the summary to be more specific about
the quantitative results.

Turn 3:
Add one sentence about the limitations or future work
mentioned.
```

Each turn is submitted as a separate message in the conversation history. For local models, Ollama's `/api/chat` endpoint is used with an accumulated `messages` array. For API models, the provider's native chat interface is used with the same message structure. The full conversation state (including all prior turns) is hashed and logged as `conversation_history_hash` in the Run Card.

S1.4 Task 4: RAG Extraction

Listing 4: Prompt template for Task 4 (RAG extraction).

```
Using the context passage below and the scientific
abstract, extract the following fields. Return JSON only.

Context: {retrieved_passage}
Abstract: {abstract}

Fields: objective, method, key_result, model_or_system,
benchmark

JSON:
```

This task augments the structured extraction prompt (Task 2) with a retrieved context passage prepended to the abstract. The context passage is a domain-relevant text segment that provides additional information about the paper's topic. The purpose is to test whether augmenting the prompt with external context affects reproducibility.

S2 Input Abstracts

The experiment corpus comprises 30 scientific abstracts drawn from highly cited publications in machine learning, computer vision, natural language processing, and reinforcement learning. Table S1 lists all abstracts by number, authors, year, title, and venue. Full text and DOIs are available in the repository at `data/inputs/abstracts.json`.

Table S1: Input abstracts used in all experiments (30 total).

#	Citation
1	Vaswani et al. (2017) – Attention Is All You Need, NeurIPS
2	Devlin et al. (2019) – BERT: Pre-training of Deep Bidirectional Transformers, NAACL
3	Brown et al. (2020) – Language Models are Few-Shot Learners, NeurIPS
4	Raffel et al. (2020) – Exploring the Limits of Transfer Learning with T5, JMLR
5	Wei et al. (2022) – Chain-of-Thought Prompting Elicits Reasoning in LLMs, NeurIPS
6	Goodfellow et al. (2014) – Generative Adversarial Nets, NeurIPS

#	Citation
7	He et al. (2016) – Deep Residual Learning for Image Recognition, CVPR
8	Kingma & Welling (2014) – Auto-Encoding Variational Bayes, ICLR
9	Hochreiter & Schmidhuber (1997) – Long Short-Term Memory, Neural Computation
10	Radford et al. (2021) – Learning Transferable Visual Models (CLIP), ICML
11	Ramesh et al. (2022) – Hierarchical Text-Conditional Image Generation (DALL-E 2), arXiv
12	Rombach et al. (2022) – High-Resolution Image Synthesis with Latent Diffusion Models, CVPR
13	Touvron et al. (2023) – LLaMA: Open and Efficient Foundation Language Models, arXiv
14	Ouyang et al. (2022) – Training Language Models to Follow Instructions (InstructGPT), NeurIPS
15	OpenAI (2023) – GPT-4 Technical Report, arXiv
16	Chowdhery et al. (2022) – PaLM: Scaling Language Modeling with Pathways, JMLR
17	Liu et al. (2019) – RoBERTa: A Robustly Optimized BERT Pretraining Approach, arXiv
18	Lewis et al. (2020) – BART: Denoising Sequence-to-Sequence Pre-training, ACL
19	Dosovitskiy et al. (2021) – An Image is Worth 16x16 Words (ViT), ICLR
20	Kirillov et al. (2023) – Segment Anything, ICCV
21	Ho et al. (2020) – Denoising Diffusion Probabilistic Models, NeurIPS
22	Schulman et al. (2017) – Proximal Policy Optimization Algorithms, arXiv
23	Silver et al. (2016) – Mastering the Game of Go (AlphaGo), Nature
24	Kipf & Welling (2017) – Semi-Supervised Classification with GCNs, ICLR
25	Mikolov et al. (2013) – Efficient Estimation of Word Representations (Word2Vec), ICLR Workshop
26	Bahdanau et al. (2015) – Neural Machine Translation by Jointly Learning to Align and Translate, ICLR
27	Rezende & Mohamed (2015) – Variational Inference with Normalizing Flows, ICML
28	Zoph & Le (2017) – Neural Architecture Search with Reinforcement Learning, ICLR
29	Hu et al. (2022) – LoRA: Low-Rank Adaptation of Large Language Models, ICLR
30	Touvron et al. (2023) – Llama 2: Open Foundation and Fine-Tuned Chat Models, arXiv

The corpus spans publication years from 1997 to 2023 and includes foundational works (Transformers, GANs, LSTM), scaling studies (GPT-3, PaLM, T5), alignment methods (InstructGPT, RLHF), efficient adaptation (LoRA), and multimodal models (CLIP, DALL-E 2, SAM). This diversity ensures that our findings are not artefacts of a single subfield or writing style.

S3 Retrieved Contexts for RAG

For Task 4 (RAG Extraction), each of the 30 input abstracts is augmented with a domain-relevant context passage prepended to the prompt. These passages serve as simulated retrieval results, providing additional topical information that the model can use when extracting structured fields.

The context passages were curated to satisfy two criteria: (1) domain relevance to the target abstract's topic (e.g., a passage about attention mechanisms for the Transformer abstract), and (2) non-overlap with the abstract text itself, to test whether the model integrates external context

consistently across repeated runs.

Each context passage is approximately 150–300 words in length and is drawn from survey papers, textbook descriptions, or related work sections of the cited papers. The exact text of all 30 context passages, along with SHA-256 hashes verifying their integrity, is available in the project repository at `data/inputs/rag_contexts.json`.

The key finding for RAG tasks is that context augmentation does not substantially improve API reproducibility: GPT-4 achieves EMR = 0.230 on RAG extraction (compared to 0.443 on standard extraction under C1), and Gemini 2.5 Pro achieves EMR = 0.070. Local models maintain near-perfect reproducibility regardless of context augmentation (see main text, Table 1).

S4 API Payload Documentation

To address potential "apples-to-oranges" concerns regarding differences in API request structures across providers, we document the exact JSON payload structures sent to each of the seven inference endpoints. All payloads were constructed deterministically and logged as part of the Run Card. Hyperlinks to the official API references for each provider are listed in Table S2.

Comparability across providers. Providers expose overlapping but not identical parameter surfaces. What is comparable across stacks is therefore not parameter *equality* but the operational claim that we used the *maximum determinism configuration each provider documents*: temperature zero is universally exposed; the seed parameter is supported by OpenAI (advisory) and Gemini, absent from the Anthropic Messages API surface, and not exposed by DeepSeek's OpenAI-compatible Chat Completions interface or by Perplexity Sonar. Our `seed_status` Run Card field records this asymmetry per stack as `sent`, `logged-only-not-sent`, or `not-supported`. The precise per-stack parameter set for every payload is in the listings below.

Table S2: Official API documentation for each provider used in this study (accessed 2026).

Provider / endpoint	**Reference URL**
OpenAI Chat Completions	https://platform.openai.com/docs/api-reference/chat
Anthropic Messages	https://docs.anthropic.com/en/api/messages
Google Gemini (AI Studio REST)	https://ai.google.dev/api/rest
DeepSeek Chat	https://api-docs.deepseek.com
Perplexity Sonar	https://docs.perplexity.ai
Together AI Chat Completions	https://docs.together.ai/reference/chat-completions
Ollama (local)	https://github.com/ollama/ollama/blob/main/docs/api.md

The "Key difference" callouts in the per-provider subsections that follow refer to provider-specific deviations from the canonical OpenAI Chat Completions request schema.

S4.1 Ollama (Local Models)

Single-turn tasks (Tasks 1–2) use `POST /api/generate`:

Listing 5: Ollama generate payload (Tasks 1–2).

```
{
  "model": "llama3:8b",
  "prompt": "<full prompt text>",
  "options": {
    "temperature": 0.0,
```

```
    "seed": 42,
    "num_predict": 1024
  },
  "stream": false
}
```

The `model` field is set to `llama3:8b`, `mistral:7b`, or `gemma2:9b` as appropriate. Multi-turn tasks (Task 3) use `POST /api/chat` with an accumulated `messages` array containing alternating `user` and `assistant` roles. No system prompt, stop sequences, or post-processing are applied.

S4.2 OpenAI (GPT-4)

Accessed via the `openai` Python SDK v1.59.9:

Listing 6: OpenAI Chat Completions payload.

```
{
  "model": "gpt-4",
  "messages": [
    {"role": "user", "content": "<prompt>"}
  ],
  "temperature": 0.0,
  "seed": 42,
  "max_tokens": 1024
}
```

No system message, stop sequences, `top_p`, `frequency_penalty`, or `presence_penalty` were set (all defaults). The resolved model version (`gpt-4-0613`) was extracted from the response object and logged. OpenAI documents that the `seed` parameter provides "mostly deterministic" outputs but does not guarantee bitwise reproducibility.

S4.3 Anthropic (Claude Sonnet 4.5)

Accessed via `urllib` (no SDK dependency):

Listing 7: Anthropic Messages API payload.

```
{
  "model": "claude-sonnet-4-5-20250929",
  "messages": [
    {"role": "user", "content": "<prompt>"}
  ],
  "temperature": 0.0,
  "max_tokens": 1024
}
```

Key difference: The Anthropic API does not support a `seed` parameter. The seed value in the Run Card is marked `seed_status: "logged-only-not-sent-to-api"`. No system message or stop sequences were used.

S4.4 Google (Gemini 2.5 Pro)

Accessed via `urllib` (no SDK dependency) through the Google AI Studio REST API:

Listing 8: Google Gemini API payload.

```
{
  "contents": [
    {
      "role": "user",
      "parts": [{"text": "<prompt>"}]
    }
  ],
  "generationConfig": {
    "maxOutputTokens": 8192,
    "temperature": 0.0,
    "seed": 42
  },
  "systemInstruction": {
    "parts": [{"text": "<system>"}]
  }
}
```

Key difference: The `seed` parameter is supported by the Gemini API and is sent with every request. Gemini 2.5 Pro is a "thinking" model: internal reasoning tokens consume the `maxOutputTokens` budget, hence the higher limit (8,192 vs. 1,024 for other models). Multi-turn tasks use the same endpoint with an accumulated `contents` array alternating `role: "user"` and `role: "model"`.

S4.5 DeepSeek (DeepSeek Chat)

Accessed via the OpenAI-compatible API:

Listing 9: DeepSeek Chat API payload.

```
{
  "model": "deepseek-chat",
  "messages": [
    {"role": "user", "content": "<prompt>"}
  ],
  "temperature": 0.0,
  "max_tokens": 1024
}
```

No seed parameter, system message, or stop sequences. DeepSeek Chat achieved the highest API reproducibility (EMR = 0.800 for extraction), suggesting effective internal determinism under greedy decoding.

S4.6 Perplexity (Perplexity Sonar)

Accessed via the Perplexity API:

Listing 10: Perplexity Sonar API payload.

```
{
  "model": "sonar",
  "messages": [
    {"role": "user", "content": "<prompt>"}
  ],
  "temperature": 0.0,
  "max_tokens": 1024
```

```
}
```

Perplexity Sonar is an online model with real-time search augmentation. No seed parameter or system message. The search-augmented nature introduces an additional source of variability: retrieved web content may differ across requests, contributing to the lowest observed reproducibility (EMR = 0.010–0.100).

S4.7 Together AI (LLaMA 3 8B)

Accessed via the Together AI OpenAI-compatible API:

Listing 11: Together AI API payload.

```
{
  "model": "meta-llama/Meta-Llama-3-8B-Instruct",
  "messages": [
    {"role": "user", "content": "<prompt>"}
  ],
  "temperature": 0.0,
  "seed": 42,
  "max_tokens": 1024
}
```

Together AI serves the same open-weight LLaMA 3 8B model used locally, enabling a quasi-isolation probe: differences in EMR between local and cloud-served deployments of the same weights can be attributed to infrastructure effects (tensor parallelism, dynamic batching) rather than model architecture or training differences.

S4.8 Key Symmetry Points

Across all nine model deployments: (1) identical prompt text (verified by `prompt_hash`); (2) identical temperature ($t = 0.0$); (3) identical maximum token limit (1,024; 8,192 for Gemini 2.5 Pro to accommodate thinking tokens); (4) no system messages for single-turn tasks; (5) no stop sequences; (6) no post-processing or text normalisation of outputs.

S5 Protocol Comparison Table

Table S3 provides a systematic feature-by-feature comparison of our protocol with five existing experiment-tracking and evaluation tools. The key distinction is not merely one of tooling but of scientific capability: existing tools log what happened during training (parameters, metrics, artefacts), whereas our protocol enables answering questions about whether two generative outputs are provably derived from identical configurations, which exact factor caused a divergence, and whether an output has been tampered with post-generation.

Table S3: Comparison of our protocol with existing reproducibility tools and frameworks. Checkmarks indicate full support; tildes (∼) indicate partial support; dashes (–) indicate no support.

Feature	Ours	MLflow	W&B	DVC	OAI Evals	LangSmith
Prompt versioning	✓	–	∼	–	∼	∼
Output hashing	✓	–	–	✓	–	–
PROV integration	✓	–	–	–	–	–
Env. fingerprinting	✓	∼	∼	∼	–	–
Overhead <1%	✓	∼	∼	N/A	N/A	∼
Inference-specific design	✓	–	–	–	✓	✓
Open standard	✓	–	–	–	–	–
Seed & param logging	✓	✓	✓	–	✓	✓
Model weights hashing	✓	–	∼	✓	–	–
Local-first (no cloud)	✓	✓	–	✓	–	–

OAI Evals = OpenAI Evals; W&B = Weights & Biases. Assessment based on publicly documented features as of February 2026.

MLflow provides experiment tracking, model packaging, and deployment. It logs parameters, metrics, and artefacts, but focuses on training pipelines and numerical outcomes rather than text-generation provenance.
Weights & Biases offers experiment tracking with visualisation dashboards. It supports prompt logging but lacks structured prompt versioning, cryptographic output hashing, and provenance graph generation.
DVC provides data versioning through git-like operations. While effective for dataset management, it does not address run-level provenance or prompt documentation.
OpenAI Evals is a framework for evaluating LLM outputs against benchmarks. It provides structured evaluation but is tightly coupled to OpenAI's ecosystem and does not generate interoperable provenance records.
LangSmith offers tracing and evaluation for LLM applications. It captures detailed execution traces but uses a proprietary format and requires cloud connectivity.

S6 Protocol Minimality – Ablation Study

To substantiate the protocol's minimality claim, we systematically removed each field group from the Run Card schema and assessed which audit questions became unanswerable. The analysis was conducted over 1,864 run records.

S6.1 Field Groups and Audit Questions

We decomposed the Run Card into eight field groups:

1. **Identification**: `run_id`, `task_category`, `task_id`, `interaction_regime`
2. **Model Context**: `model_name`, `model_version`, `model_source`, `weights_hash`
3. **Parameters**: `inference_params`, `params_hash`
4. **Input/Output**: `prompt_text`, `input_text`, `input_hash`, `output_text`, `output_hash`, `output_metrics`
5. **Hashing**: `prompt_hash`, `input_hash`, `output_hash`, `params_hash`, `environment_hash`
6. **Environment**: `environment`, `environment_hash`, `code_commit`
7. **Timing**: `timestamp_start`, `timestamp_end`, `execution_duration_ms`, `logging_overhead_ms`
8. **Overhead**: `logging_overhead_ms`, `storage_kb`

We defined 10 audit questions that a reproducibility-oriented researcher might ask:

Q1. Can we verify the exact prompt used?
Q2. Can we verify the model identity?
Q3. Can we verify the generation parameters?
Q4. Can we detect output tampering?
Q5. Can we verify the execution environment?
Q6. Can we reproduce the exact timing?
Q7. Can we assess protocol overhead?
Q8. Can we trace full provenance?
Q9. Can we verify input integrity?
Q10. Can we compare outputs across runs?

S6.2 Ablation Matrix

Table S4 shows the ablation results. Each × indicates that removing the corresponding field group renders the audit question unanswerable.

Table S4: Ablation matrix: which audit questions (Q1–Q10) become unanswerable when each field group is removed. × = question becomes unanswerable.

Field Group	Q1	Q2	Q3	Q4	Q5	Q6	Q7	Q8	Q9	Q10	**Lost**
Identification								×		×	2
Model Context		×								×	2
Parameters			×							×	2
Input/Output	×			×					×	×	4
Hashing	×		×	×	×				×	×	6
Environment					×						1
Timing						×	×				2
Overhead							×				1

The Hashing group affects 6/10 questions despite contributing only ∼410 bytes per run (<10% of the record). Every row has at least one ×, confirming minimality: no field group can be removed without losing audit capability.

S6.3 Storage and Overhead Analysis

Table S5 reports the storage cost and computational overhead of each field group.

Table S5: Storage and overhead per field group, measured across 1,864 run records.

Field Group	**Mean bytes**	**% of record**	**Est. ms**	**Verdict**
Identification	158.6	3.8%	0.0	Essential (low)
Model Context	106.1	2.6%	0.0	Essential (low)
Parameters	206.5	5.0%	1.3	Essential (low)
Input/Output	2,413.8	58.6%	0.0	Essential (med)
Hashing	409.0	9.9%	10.2	Essential (high)
Environment	547.9	13.3%	8.9	Essential (low)
Timing	163.8	4.0%	2.5	Essential (low)
Overhead	45.8	1.1%	2.5	Essential (low)
Total	∼4,052	100%	∼25	

Total overhead per run: approximately 4,052 bytes storage and 25 ms computation, representing <1% of typical inference time.

S7 Chat-Format Control Experiment

To assess whether the prompt-format difference between completion-style and chat-style API calls contributes to the observed reproducibility gap between local and API models, we conducted a control experiment running LLaMA 3 8B through two different Ollama endpoints.

S7.1 Design

- **Completion format**: `POST /api/generate` (raw prompt string)
- **Chat format**: `POST /api/chat` (structured messages with `role: "user"`)
- **Scale**: 10 abstracts × 2 tasks (summarization, extraction) × 2 conditions (C1 fixed seed, C2 variable seed) × 5 repetitions = 200 runs per format, 400 runs total
- **Decoding**: Greedy ($t = 0$, seed = 42 for C1)

S7.2 Results

Table S6 presents the per-format metrics. The two formats produce identical aggregate metrics for both tasks.

Table S6: Chat-format control experiment: LLaMA 3 8B, completion vs. chat format (200 runs each).

Task	Format	EMR	NED	ROUGE-L
Summarization	Completion	0.929	0.007	0.992
Summarization	Chat	0.929	0.007	0.992
Extraction	Completion	1.000	0.000	1.000
Extraction	Chat	1.000	0.000	1.000

EMR = Exact Match Rate (fraction of abstract groups where all 5 repetitions are identical); NED = Normalized Edit Distance; ROUGE-L = longest common subsequence F-measure.

Per-abstract EMR values are also identical across formats for all 10 abstracts in both tasks. For summarization, abstracts 1 and 3 show EMR = 0.644 in both formats (indicating some within-group variation attributable to the warm-up effect on the first repetition), while all other abstracts achieve EMR = 1.000 in both formats. For extraction, all 10 abstracts achieve EMR = 1.000 in both formats.

S7.3 Interpretation

The chat-format control provides evidence that prompt formatting is **not** a source of the reproducibility gap between local and API models. The identical metrics across completion and chat formats indicate that the local model produces the same outputs regardless of whether the prompt is presented as a raw string or as a structured chat message. This result supports the interpretation that the local–API gap is driven by infrastructure-level factors (tensor parallelism, dynamic batching, mixed-precision non-determinism) rather than by differences in prompt formatting conventions.

S8 Reproducibility Checklist

The following 15-item checklist is designed for self-assessment of reproducibility in generative AI studies. Each item maps to a specific field or artefact in our protocol. We organise the checklist into four categories.

Category 1: Prompt Documentation (4 items)

1. Is the exact prompt text recorded and versioned? [`Prompt Card: prompt_text, prompt_hash`]
2. Are design assumptions and limitations documented? [`Prompt Card: assumptions, limitations`]
3. Is the expected output format specified? [`Prompt Card: expected_output_format`]
4. Is the interaction regime documented (single-turn/multi-turn)? [`Prompt Card: interaction_regime`]

Category 2: Model and Environment (4 items)

5. Is the model name and version recorded? [`Run Card: model_name, model_version`]
6. Are model weights hashed for identity verification? [`Run Card: weights_hash`]
7. Is the execution environment fingerprinted? [`Run Card: environment, environment_hash`]
8. Is the source code version recorded? [`Run Card: code_commit`]

Category 3: Execution and Output (5 items)

9. Are all inference parameters logged? [`Run Card: inference_params`]
10. Is the random seed recorded? [`Run Card: inference_params.seed`]
11. Is the output cryptographically hashed? [`Run Card: output_hash`]
12. Are execution timestamps recorded? [`Run Card: timestamp_start, timestamp_end`]
13. Is logging overhead measured separately? [`Run Card: logging_overhead_ms`]

Category 4: Provenance (2 items)

14. Is a provenance graph generated per experiment group? [`PROV-JSON document`]
15. Are provenance documents in an interoperable format? [`W3C PROV standard`]

A study satisfying all 15 items achieves the highest level of reproducibility documentation under our framework. We recommend that studies report at minimum items 1, 5, 9, 10, and 11 (see main text for discussion).

S9 Statistical Test Results

Section at a glance. *Of 68 pre-registered pairwise tests (Fisher's exact, Mann–Whitney U, Brunner–Munzel, plus paired follow-ups),* ***51 remain significant after Holm–Bonferroni control at*** $\alpha = 0.05$*. Cliff's* δ *effect sizes for the local–API contrasts fall in the range* ***0.784–0.896 (large to very large, Romano–Vargha–Delaney bands)****, indicating that the EMR gap is not a borderline effect but a categorical separation between the two stack classes. The remainder of this section reports the procedure, the per-comparison results, and the effect-size table that supports this summary.*

S9.1 Methodology

We applied the Holm–Bonferroni sequential rejection procedure to control the family-wise error rate at $\alpha = 0.05$ across all pairwise comparisons. The test battery comprised 68 individual tests:

- 24 Fisher's exact tests (3 local models × 4 API models × 2 tasks): testing whether the proportion of abstracts with perfect reproducibility (EMR = 1.0) differs between local and API models.
- 24 Mann–Whitney U tests (same pairings): testing whether per-abstract EMR distributions differ between local and API models.
- 18 Wilcoxon signed-rank tests (3 local × 4 API × 2 tasks, where paired data available on shared abstracts): testing within-abstract EMR differences.
- 2 aggregate Mann–Whitney U tests (all local vs. all API, per task): testing the overall local–API gap.

S9.2 Holm–Bonferroni Procedure

The Holm–Bonferroni method orders all $m = 68$ p-values from smallest to largest and rejects hypothesis $H_{(k)}$ if $p_{(k)} \leq \alpha/(m - k + 1)$ for all $k' \leq k$. This provides strong control of the family-wise error rate without assuming independence among tests.

Scope of the FWER family. The 68-test family corresponds to the pre-specified comparisons on Tasks 1–2 (extraction and summarisation). The code, mathematics, and cross-domain results (HumanEval, GSM8K, PubMed $PM_{2.5}$, and the multi-turn extension to gpt-4o and DeepSeek) are reported descriptively with 95% bootstrap confidence intervals; they are not included in the Holm–Bonferroni family because the qualitative comparisons of interest (local-vs.-API gap, Claude-as-outlier pattern) are confirmatory rather than exploratory under the pre-specified framework, and adding them post hoc would inflate the family-wise error rate of the pre-specified tests. Readers wishing to apply a unified correction across the full design can use the per-stack per-task EMR with 95% CIs reported in the main manuscript (Table 4, "Exact match rate (EMR) with 95% bootstrap CI per deployment stack on the code, mathematics, and cross-domain tasks").

Bootstrap behaviour at the EMR boundary. The percentile bootstrap was selected for consistency across all analyses, but it is known to produce degenerate intervals when the empirical estimate sits at the boundary of the EMR support (0 or 1). For groups with EMR=1.000 the percentile CI collapses to $[1.00, 1.00]$; for groups with EMR=0.000 it collapses to $[0.00, 0.00]$. These "zero-width" intervals understate uncertainty by design and are reported as such in the tables. Bias-corrected and accelerated (BCa) intervals would marginally widen the boundary CIs but do not change any of the qualitative conclusions in this paper.

S9.3 Summary Results

Of the 68 tests, **51 were rejected** (significant) and 17 were not rejected after correction:

- All 24 Fisher's exact tests involving LLaMA 3 8B, Gemma 2 9B, and Mistral 7B versus GPT-4, Claude, and Perplexity were significant.
- All aggregate local-vs.-API tests were significant ($p_{\text{adj}} < 10^{-9}$).
- The 17 non-rejected tests primarily involved comparisons with DeepSeek Chat, which has the highest API reproducibility (EMR = 0.800 for extraction), making the gap with some local models non-significant at the corrected threshold.

S9.4 Representative Results

Table S7 presents a selection of representative Holm–Bonferroni-corrected results spanning the range from the most significant to the borderline cases.

Table S7: Representative Holm–Bonferroni-corrected test results (selection of 16 from 68 total). Rank = position in the sorted p-value list.

Test	Rank	p_{orig}	p_{adj}	Reject
Fisher: LLaMA vs. GPT-4, Extr.	2	<0.001	<0.001	Yes
Fisher: LLaMA vs. Perplexity, Extr.	3	<0.001	<0.001	Yes
Fisher: Gemma vs. GPT-4, Summ.	4	<0.001	<0.001	Yes
Fisher: LLaMA vs. GPT-4, Summ.	5	<0.001	<0.001	Yes
Aggregate: Local vs. API, Summ.	6	9.3×10^{-17}	5.9×10^{-15}	Yes
Aggregate: Local vs. API, Extr.	7	1.0×10^{-14}	6.4×10^{-13}	Yes
MW: LLaMA vs. GPT-4, Summ.	8	4.2×10^{-11}	2.6×10^{-9}	Yes
Fisher: Gemma vs. Claude, Extr.	33	0.00012	0.0043	Yes
Fisher: Mistral vs. Claude, Extr.	47	0.0011	0.024	Yes
Wilcoxon: Mistral vs. Claude, Summ.	48	0.0020	0.041	Yes
Fisher: LLaMA vs. DeepSeek, Extr.	50	0.0020	0.039	Yes
MW: Gemma vs. DeepSeek, Summ.	51	0.0025	0.045	Yes
Fisher: LLaMA vs. DeepSeek, Summ.	53	0.0074	0.118	No
Fisher: Gemma vs. DeepSeek, Extr.	61	0.033	0.260	No
MW: Mistral vs. DeepSeek, Extr.	62	0.054	0.381	No
Fisher: Mistral vs. DeepSeek, Summ.	68	0.656	0.656	No

MW = Mann–Whitney U test. All p-values are two-sided. The complete set of 68 tests is available in the repository at `analysis/enhanced_statistical_results.json`.

S9.5 Effect Sizes

Cohen's h effect sizes for all pairwise Fisher comparisons ranged from 0.40 (Mistral vs. DeepSeek, summarization – medium effect) to 3.14 (Gemma vs. Claude/Perplexity, summarization – very large effect). All comparisons involving Gemma 2 9B vs. any API model yielded very large effect sizes ($h > 1.5$), reflecting Gemma's perfect reproducibility (EMR = 1.000) against API models' substantially lower rates.

S10 Experimental Coverage Matrix

Table S8 reports the number of abstracts and total runs per model–task–condition combination. The study encompasses 3,904 experimental runs across 9 model deployments, 4 tasks, and up to 5

experimental conditions.

Table S8: Number of abstracts (runs) per model–task–condition. Dash = not evaluated.

Model	Task	C1	C2	C3 (t=0)	C3 (t=0.3)	C3 (t=0.7)
LLaMA 3 8B	Extr.	30 (150)	30 (150)	30 (90)	30 (90)	30 (90)
	Summ.	30 (150)	30 (150)	30 (90)	30 (90)	30 (90)
	Multi	10 (50)	–	–	–	–
	RAG	10 (50)	–	–	–	–
Mistral 7B	Extr.	10 (50)	10 (50)	10 (30)	10 (30)	10 (30)
	Summ.	10 (50)	10 (50)	10 (30)	10 (30)	10 (30)
	Multi	10 (50)	–	–	–	–
	RAG	10 (50)	–	–	–	–
Gemma 2 9B	Extr.	10 (50)	10 (50)	10 (30)	10 (30)	10 (30)
	Summ.	10 (50)	10 (50)	10 (30)	10 (30)	10 (30)
	Multi	10 (50)	–	–	–	–
	RAG	10 (50)	–	–	–	–
GPT-4	Extr.	–	30 (150)	17 (51)	17 (51)	14 (42)
	Summ.	3 (8)*	30 (150)	30 (90)	30 (90)	30 (90)
Claude 4.5	Extr.	10 (49)†	10 (50)	10 (30)	10 (30)	10 (30)
	Summ.	10 (50)	10 (50)	10 (30)	10 (30)	10 (30)
	Multi	10 (50)	–	–	–	–
	RAG	10 (50)	–	–	–	–
Gemini 2.5 Pro	Multi	10 (50)	–	–	–	–
	RAG	10 (50)	–	–	–	–
DeepSeek Chat	Extr./Summ.	10 (100)	–	–	–	–
Perplexity Sonar	Extr./Summ.	10 (100)	–	–	–	–
Together AI LLaMA	Extr./Summ.	10 (100)	10 (100)	–	–	–

*GPT-4 C1 summarization: only 3 abstracts (8 runs) before API quota exhaustion. †Claude C1 extraction: 49 runs (1 API timeout returned empty output). An additional 200 chat-format control runs (LLaMA 3 via `/api/chat`) are documented in Section S7 but not shown here.

S10.1 Experimental Conditions

The five experimental conditions are:

C1 (Fixed seed): $t = 0$, seed = 42, 5 repetitions per abstract. Tests baseline reproducibility under maximal determinism settings.

C2 (Variable seed): $t = 0$, seeds $\in \{42, 123, 456, 789, 1024\}$, 5 repetitions per abstract. Tests whether changing the seed affects output under greedy decoding.

C3 ($t = 0.0$): Explicit temperature 0 with 3 repetitions using different seeds. Overlaps with C1/C2 to verify consistency.

C3 ($t = 0.3$): Low temperature with 3 repetitions. Tests near-greedy regime.

C3 ($t = 0.7$): Moderate temperature with 3 repetitions. Tests effect of increased stochasticity.

S10.2 Total Runs Summary

Table S9: Total runs by model category.

Category	Runs
Local models (LLaMA, Mistral, Gemma)	2,400
API models (GPT-4, Claude, Gemini, DeepSeek, Perplexity)	1,304
Cloud-served open-weight (Together AI)	200
Grand total	**3,904**

Excludes 200 chat-format control runs (see Section S7). Including controls, the total is 4,104 runs.

S11 Code, Mathematics, and Cross-Domain Tasks: Experimental Coverage

Section at a glance. *Beyond Tasks 1–4, the study includes* ***2,900 further controlled experiments*** *(total 7,004) covering three additional task families—****HumanEval (code), GSM8K (math), and a cross-domain PubMed $PM_{2.5}$ extraction probe****—plus a multi-turn extension to gpt-4o and DeepSeek. The headline finding is that the local–API reproducibility gap* ***persists and widens precisely on high-stakes tasks where small textual differences alter functionality or numerical correctness****: Claude Sonnet 4.5 reaches EMR = 0.063 [0.03, 0.10] on GSM8K math reasoning and 0.010 [0.00, 0.03] on PubMed $PM_{2.5}$ extraction, while local stacks maintain EMR $\geq$ 0.92 in every one of these tasks. The pattern is* domain-transferable*, not corpus-specific.*

This section documents the code, mathematics, and cross-domain experiments and their coverage.

S11.1 Additional task families

Three additional task families and one targeted extension were evaluated under the same C1 condition (t=0, fixed seed = 42, 5 repetitions per problem):

HumanEval (code generation). We sampled 30 problems stratified by docstring length (proxy for difficulty) from the canonical 164-problem benchmark of Chen et al. (2021). Each problem provides a Python function signature with a docstring; the model must emit a syntactically valid function body. Repetitions are SHA-256 hashed and compared at the source-code level (EMR), along with NED, ROUGE-L, and BERTScore F1. The 30-problem subset is fixed and cached at `data/inputs/revision/humaneval.jsonl`.

GSM8K (math reasoning). We sampled 30 problems uniformly at random (seed = 42) from the 1,319-problem test split of Cobbe et al. (2021). Each problem provides a grade-school math word problem with a chain-of-thought solution terminated by `#### <integer>`. We hash and compare the full output (chain-of-thought plus answer). Cached at `data/inputs/revision/gsm8k_test.jsonl`.

PubMed $PM_{2.5}$/respiratory health (out-of-AI/ML structured extraction). Ten abstracts on fine particulate matter and respiratory outcomes were drawn from the companion-paper corpus (Rover & de Souza Tadano, 2026, OSF: https://doi.org/10.17605/OSF.IO/VR934). They are independent of the 30 AI/ML abstracts used for Tasks 1–4. The extraction prompt is identical in form to Task 2 but applied to a different scientific domain to test cross-domain transfer of the reproducibility gap. Cached at `data/inputs/revision/pubmed_pm25_t14.json`.

Multi-turn extension to gpt-4o and DeepSeek Chat. The three-turn refinement protocol from Task 3 was applied to the gpt-4o and DeepSeek Chat stacks (10 abstracts × 5 reps each, 50 runs per stack) to test universality of the near-zero multi-turn EMR observed for Claude and Gemini.

S11.2 Run totals for the additional tasks

Table S10: Run counts for the code, mathematics, and cross-domain tasks. All runs are under C1 ($t = 0$, seed = 42, 5 reps).

Task	Stacks	Runs
HumanEval (30 problems)	8	1,200
GSM8K (30 problems)	8	1,200
PubMed $PM_{2.5}$ (10 abstracts)	8	400
Multi-turn extension (10 abstracts)	2 (gpt-4o, deepseek-chat)	100
Subtotal		**2,900**
Combined total (4,104 + 2,900)		**7,004**

All 2,900 runs use the Run Card protocol (Methods §Protocol design); per-run JSONs are deposited in `outputs/revision/runs/`.

S11.3 Reproducibility infrastructure

A unified runner with a resumable orchestrator executes these tasks. The pipeline shares the existing Run Card + PROV instrumentation (Methods §Protocol design) and supports checkpoint-based resume across sessions. Code-generation outputs are evaluated in a sandboxed subprocess (5-second wall clock, POSIX resource limits) for pass@1; the sandbox is documented at `src/tasks/pass_at_1.py`.

S11.4 Snapshot governance

The OpenAI stack for the code, mathematics, and cross-domain tasks resolves to `gpt-4o-2024-11-20` (most recent snapshot at the time of measurement). Tasks 1–4 used `gpt-4-0613`; both are reported and the snapshot identifier is logged per run for point-in-time auditability.

S12 LLM-as-Judge Triangulation Protocol

Section at a glance. *A blinded* ***two-judge LLM-as-judge protocol*** *(Claude Opus 4.7 from Anthropic and gpt-4o from OpenAI—two distinct training families) classified* ***30 newly sampled disagreement cases*** *against three pre-registered criteria (direction, magnitude ±20 %, CI overlap).* ***Both judges classify the majority as truly contradictory****: Claude 22/30 (Wilson 95 % CI 56–86 %); gpt-4o 27/30 (74–97 %). Inter-judge* ***Cohen's*** $\kappa = 0.29$ ***(fair agreement, Landis–Koch).*** *Both judges' lower bounds exclude a pure-cosmetic interpretation under BERTScore F1 > 0.97 on the same outputs, and the convergence of two independent provider families addresses the residual "LLM-on-LLM" concern more directly than a single-judge design would.* ***The in-paper protocol is self-sufficient to support the truly-contradictory claim****; the companion paper is cited only as larger-corpus context.*

This section documents the in-paper triangulation of the applied $PM_{2.5}$ divergence claim. The validation is layered with the companion paper (Rover & de Souza Tadano, 2026, OSF: https:

//doi.org/10.17605/OSF.IO/VR934) to preserve that paper's contributions while providing self-contained evidence within the present manuscript.

S12.1 Sampling design

We sampled 30 disagreement cases from the companion-paper extraction corpus (`extraction_long.json`, 21,799 extractions), distinct from the 23 study-level effect estimates analysed in the companion paper. Cases were drawn in two waves: an initial 10 cases (seed = 42) followed by 20 additional cases (seed = 4242) to extend the sample for inter-judge analysis. Cases were drawn from the three cloud stacks where disagreement is highest (Claude Sonnet 4.5, Gemini 2.5 Pro, GPT-4.1; per the companion paper's pairwise-disagreement analysis). Stratification was applied across both waves to balance across stack and across kind of disagreement (direction, magnitude, CI overlap, mixed).

A disagreement case is a pair of runs (within the same stack, the same input abstract, and the same estimate index) whose effect estimate, CI bounds, or sign differs beyond pre-registered thresholds. Final composition: 18 Claude Sonnet 4.5 cases and 12 GPT-4.1 cases (Gemini had insufficient strictly disagreeing pairs under our thresholds in either wave), distributed across direction, magnitude, CI overlap, and mixed strata.

S12.2 Judges and prompt

Two independent LLM judges from distinct provider families were used: Claude Opus 4.7 (model identifier `claude-opus-4-7`, Anthropic Messages API) and gpt-4o (`gpt-4o-2024-11-20`, OpenAI Chat Completions API). Each judge received the same system prompt instructing pre-registered evaluation along three criteria:

1. **Direction.** Do both extractions report the same direction of effect? (positive, null, negative)
2. **Magnitude.** Do the effect estimates agree within ±20% relative to their average?
3. **CI overlap.** If both 95% CIs are reported, do they share any range?

Each judge issues one verdict per case from {`truly_contradictory`, `semantically_equivalent`, `ambiguous`} plus a brief rationale. The two extractions are presented as "Extraction X" and "Extraction Y" in deterministically randomised order (per-case seed derived from SHA-256 hash of the case identifier) to prevent positional bias. Each judge is blind to which stack produced which extraction and to the other judge's verdict.

S12.3 Cost and provenance

Total cost across 30 cases × 2 judges = 60 LLM-as-judge calls: USD 0.28 (Claude Opus 4.7 on Anthropic API; mean 929 input + 137 output tokens per case at $15/MTok input, $75/MTok output) + USD 1.02 (gpt-4o on OpenAI API; mean 1,100 input + 145 output tokens per case at $2.50/MTok input, $10/MTok output) = USD 1.30. Per-case judge records (input prompt SHA-256, model identifier, randomisation seed, raw response, parsed verdict, token counts, cost) are deposited as PROV-style JSON at `outputs/revision/t3_judge/` (subdirectories `extended_claude/`, `gpt4o_original10/`, `gpt4o_extended20/`). The two-judge aggregate with Cohen's κ is at `outputs/revision/t3_j`

S12.4 Verdict distribution

Table S11: LLM-as-judge verdicts on 30 $PM_{2.5}$ disagreement cases, scored independently by two judges (Claude Opus 4.7, Anthropic; gpt-4o, OpenAI).

	Claude Opus 4.7		gpt-4o	
Verdict	**Count**	**Wilson 95% CI**	**Count**	**Wilson 95% CI**
truly_contradictory	22	[0.56, 0.86]	27	[0.74, 0.97]
semantically_equivalent	3	[0.03, 0.26]	3	[0.03, 0.26]
ambiguous	5	[0.07, 0.34]	0	—
Total cases	**30**	—	**30**	—

Inter-judge agreement: 23/30 cases (76.7%); Cohen's $\kappa = 0.29$ (fair agreement on the Landis–Koch scale). The seven disagreements are concentrated where Claude returns *ambiguous* and gpt-4o returns *truly_contradictory*, indicating direction agreement (both judges classify the majority as substantive disagreement) with Claude more conservative on borderline cases. The 30-case sample comprises 18 Claude Sonnet 4.5 and 12 GPT-4.1 cases, stratified across direction, magnitude, CI overlap, and mixed disagreement kinds; the first 10 cases (seed=42) and the additional 20 (seed=4242) are independently sampled from the companion-paper corpus and exclude the 23 study-level cases of the companion paper. Total cost: USD 0.28 (Claude Opus on 30 cases) + USD 1.02 (gpt-4o on 30 cases) = USD 1.30.

The majority of divergences are substantive contradictions despite BERTScore F1 > 0.97 on the same outputs (Claude 73%, gpt-4o 90%). Consistent with the per-field analysis (Section S13), this confirms that aggregate semantic-similarity metrics are structurally unable to detect substantive payload divergence.

S12.5 What is *not* reproduced here from the companion paper

To preserve the contributions of the companion paper (Rover & de Souza Tadano, 2026, OSF: https://doi.org/10.17605/OSF.IO/VR934), the present supplementary section does *not* reproduce: (i) the 6-model × 10-run pairwise disagreement matrix; (ii) the silver-standard cross-validation against DeepSeek-R1 reasoning; (iii) the random-effects per-run pooled risk-ratio table; (iv) the small-literature meta-analytic simulation; (v) the 35 individual blindage analyses; (vi) the work-saved analysis. Readers seeking these are referred to Rover & de Souza Tadano (2026, OSF preregistration: https://doi.org/10.17605/OSF.IO/VR934).

S12.6 Future work: multi-rater human validation

To confirm whether the $PM_{2.5}$ divergences are truly contradictory or merely semantically varied, we conducted a cross-provider two-judge LLM-as-judge analysis (Claude Opus 4.7 from Anthropic and gpt-4o from OpenAI). The convergence of two judges from distinct training families directly addresses the standard "LLM-on-LLM" bias concern that a single-judge design would expose.

A dedicated methodological extension applying multi-rater human validation against a domain-expert silver standard on a larger sample is in preparation as a follow-up paper. That paper will (i) increase the sample to ≥50 cases, (ii) recruit two or more external epidemiologists not on the present author list, (iii) compute Fleiss' κ across humans, LLM judges, and the existing silver standards, and (iv) test whether the human-rater verdict distribution differs from the LLM-judge verdict distribution under formal hypothesis testing. The infrastructure for that work—blinded case packages, rating forms, and the κ analysis pipeline—is already deposited in the project repository (`human_validation/`) and is reusable by independent groups.

S13 Per-field reproducibility on structured extraction

Section at a glance. *Computing every reproducibility metric* per JSON output field *reveals an unexpected stronger argument:* ***paired Cohen's*** $d = +1.41$ ***EMR gap*** *between conclusion-relevant fields (*$\overline{EMR} = 0.455$*) and metadata fields (*$\overline{EMR} = 0.684$*)—a very large effect.* ***BERTScore F1 is structurally saturated*** *across both groups (0.978 vs 0.979, paired* $d = -0.10$*, effectively null). The most striking single-stack case is Gemini 2.5 Pro RAG on* `key_result`*: EMR = 0.10 (the field changes 90 % of the time) while BERTScore F1 remains 0.969.* ***BERTScore alone is structurally unable to discriminate substantive payload divergence from cosmetic textual variation*** *on extracted-JSON fields of moderate length; the three-level reproducibility framework (bitwise → structural → semantic) we propose is necessary.*

This section quantifies how the local–API reproducibility gap distributes across the five fields returned by the structured-extraction task (Task 2): *objective*, *method*, and *key_result* (conclusion-relevant payload) versus *model_or_system* and *benchmark* (metadata identifiers). It resolves the apparent tension between high aggregate BERTScore and the assertion that divergences affect substantive content.

S13.1 Protocol

For every (stack, abstract) group with at least two valid extractions, we computed pairwise within-group metrics on each field separately—Exact Match Rate (EMR), Normalised Edit Distance (NED), ROUGE-L F1, and BERTScore F1 (`roberta-large`, no baseline rescaling). Pairs in which at least one repetition omitted the field were skipped (rather than penalised), so the metric isolates the question "when both repetitions emit the field, how similar are they?". Per-stack point estimates are mean-of-pair-means; 95% percentile bootstrap CIs use 10,000 resamples (seed=42), matching the methodology of Section S9.

S13.2 Aggregate result

Across the six API stacks where outputs actually diverge (i.e., excluding the three local stacks with EMR=1.000 on most fields), the gap between conclusion-relevant fields and metadata fields is large on EMR but absent on BERTScore. The API-only aggregate from `analysis/bertscore_per_field_results.json`

- Conclusion-relevant fields ($\overline{\text{EMR}}$=0.455) versus metadata fields ($\overline{\text{EMR}}$=0.684). Paired Cohen's $d = +1.41$ (very large effect).
- BERTScore F1: $\overline{\text{conclusion}}$=0.978 vs $\overline{\text{metadata}}$=0.979. Paired Cohen's $d = -0.10$ (effectively null).
- NED and ROUGE-L F1 show small effects ($|d|<0.5$) in the same direction as EMR.

The single most striking single-stack illustration is **Gemini 2.5 Pro on RAG extraction, `key_result` field**: EMR=0.10 (the field changes content in 90% of pairs) while BERTScore F1 remains 0.969 on the same pairs.

S13.3 Per-stack per-field table

Table S12 (auto-generated from the same JSON file) reports per-stack per-field EMR, NED, ROUGE-L F1, and BERTScore F1 with 95% bootstrap CIs across 13 stacks and 5 fields. The pattern from the aggregate above is visible at the stack level: API stacks where outputs diverge

show conclusion-field EMR drops of 0.15–0.25 relative to metadata fields, while BERTScore is flat to within ±0.02 across field types for every stack.

S13.4 Interpretation

These results resolve the apparent contradiction between "textual not semantic" (BERTScore F1>0.97 across the board) and "affects substantive content" (the 67% `key_result` divergence rate reported in Extended Data Table 7). The resolution is that *BERTScore is structurally unable to discriminate the substantive payload divergence concentrated in conclusion-relevant fields from the cosmetic textual variation distributed across all fields* when one operates on extracted-JSON fields of moderate length. EMR can; NED partly can. The three-level reproducibility framework (bitwise → structural → semantic) that the manuscript proposes is therefore not redundant: any single metric, including a state-of-the-art semantic-similarity metric, would mislead on this corpus and at this granularity.

Table S12: Per-field reproducibility on the structured extraction task. Each cell reports the mean across abstracts with 95% percentile bootstrap CIs (n_{boot}=10,000). Pairwise comparisons within an abstract's repetitions, skipping pairs where at least one repetition omitted the field. Conclusion-relevant fields (*objective*, *method*, *key_result*) exhibit substantially lower exact-match rates than metadata fields (*model_or_system*, *benchmark*) on API stacks where outputs actually diverge (see API-only aggregate at the foot of the table). BERTScore F1, by contrast, remains saturated above 0.94 for both groups—precisely the saturation effect that motivates the manuscript's three-level reproducibility framework: high semantic similarity does not imply textual identity on the fields that drive downstream conclusions.

Stack	Field	EMR	NED	ROUGE-L F1	BERTScore F1
GPT-4	objective	0.773 [0.66, 0.87]	0.062 [0.03, 0.10]	0.947 [0.91, 0.98]	0.991 [0.98, 1.00]
	method	0.667 [0.55, 0.78]	0.088 [0.04, 0.14]	0.926 [0.88, 0.96]	0.987 [0.98, 0.99]
	key_result	0.637 [0.53, 0.74]	0.058 [0.02, 0.10]	0.952 [0.92, 0.98]	0.993 [0.99, 1.00]
	model_or_system	0.904 [0.83, 0.97]	0.022 [0.00, 0.05]	0.962 [0.93, 0.99]	0.994 [0.99, 1.00]
	benchmark	0.904 [0.82, 0.97]	0.032 [0.00, 0.07]	0.967 [0.93, 1.00]	0.995 [0.99, 1.00]
Claude Sonnet 4.5	objective	0.607 [0.41, 0.80]	0.091 [0.02, 0.18]	0.919 [0.85, 0.98]	0.988 [0.98, 1.00]
	method	0.390 [0.21, 0.61]	0.168 [0.11, 0.22]	0.838 [0.77, 0.91]	0.973 [0.96, 0.98]
	key_result	0.510 [0.34, 0.69]	0.118 [0.04, 0.20]	0.885 [0.80, 0.96]	0.981 [0.97, 0.99]
	model_or_system	0.871 [0.61, 1.00]	0.052 [0.00, 0.16]	0.922 [0.77, 1.00]	0.991 [0.97, 1.00]
	benchmark	0.738 [0.55, 0.93]	0.105 [0.03, 0.20]	0.919 [0.84, 0.98]	0.992 [0.98, 1.00]
DeepSeek Chat	objective	1.000 [1.00, 1.00]	0.000 [0.00, 0.00]	1.000 [1.00, 1.00]	1.000 [1.00, 1.00]
	method	0.840 [0.72, 0.96]	0.027 [0.00, 0.06]	0.979 [0.96, 1.00]	0.996 [0.99, 1.00]
	key_result	0.960 [0.88, 1.00]	0.007 [0.00, 0.02]	0.994 [0.98, 1.00]	0.999 [1.00, 1.00]
	model_or_system	1.000 [1.00, 1.00]	0.000 [0.00, 0.00]	1.000 [1.00, 1.00]	1.000 [1.00, 1.00]
	benchmark	1.000 [1.00, 1.00]	0.000 [0.00, 0.00]	1.000 [1.00, 1.00]	1.000 [1.00, 1.00]
Perplexity Sonar	objective	0.330 [0.21, 0.46]	0.188 [0.12, 0.26]	0.827 [0.78, 0.87]	0.973 [0.97, 0.98]
	method	0.170 [0.08, 0.29]	0.277 [0.23, 0.33]	0.714 [0.65, 0.78]	0.956 [0.94, 0.97]
	key_result	0.270 [0.18, 0.37]	0.213 [0.14, 0.28]	0.798 [0.75, 0.86]	0.970 [0.96, 0.98]
	model_or_system	0.511 [0.29, 0.73]	0.307 [0.12, 0.53]	0.626 [0.41, 0.81]	0.946 [0.90, 0.98]
	benchmark	0.300 [0.14, 0.44]	0.395 [0.26, 0.56]	0.617 [0.44, 0.76]	0.942 [0.90, 0.97]
LLaMA 3 8B	objective	0.987 [0.96, 1.00]	0.005 [0.00, 0.01]	0.996 [0.99, 1.00]	0.999 [1.00, 1.00]
	method	0.987 [0.96, 1.00]	0.005 [0.00, 0.02]	0.996 [0.99, 1.00]	1.000 [1.00, 1.00]
	key_result	0.985 [0.95, 1.00]	0.004 [0.00, 0.01]	0.996 [0.99, 1.00]	1.000 [1.00, 1.00]
	model_or_system	1.000 [1.00, 1.00]	0.000 [0.00, 0.00]	1.000 [1.00, 1.00]	1.000 [1.00, 1.00]
	benchmark	0.978 [0.93, 1.00]	0.011 [0.00, 0.03]	0.993 [0.98, 1.00]	0.999 [1.00, 1.00]
Mistral 7B	objective	1.000 [1.00, 1.00]	0.000 [0.00, 0.00]	1.000 [1.00, 1.00]	1.000 [1.00, 1.00]
	method	1.000 [1.00, 1.00]	0.000 [0.00, 0.00]	1.000 [1.00, 1.00]	1.000 [1.00, 1.00]
	key_result	0.960 [0.88, 1.00]	0.002 [0.00, 0.00]	0.999 [1.00, 1.00]	1.000 [1.00, 1.00]
	model_or_system	1.000 [1.00, 1.00]	0.000 [0.00, 0.00]	1.000 [1.00, 1.00]	1.000 [1.00, 1.00]

(continued on next page)

Table S12 continued from previous page

Stack	Field	EMR	NED	ROUGE-L F1	BERTScore F1
	benchmark	1.000 [1.00, 1.00]	0.000 [0.00, 0.00]	1.000 [1.00, 1.00]	1.000 [1.00, 1.00]
Gemma 2 9B	objective	1.000 [1.00, 1.00]	0.000 [0.00, 0.00]	1.000 [1.00, 1.00]	1.000 [1.00, 1.00]
	method	1.000 [1.00, 1.00]	0.000 [0.00, 0.00]	1.000 [1.00, 1.00]	1.000 [1.00, 1.00]
	key_result	1.000 [1.00, 1.00]	0.000 [0.00, 0.00]	1.000 [1.00, 1.00]	1.000 [1.00, 1.00]
	model_or_system	1.000 [1.00, 1.00]	0.000 [0.00, 0.00]	1.000 [1.00, 1.00]	1.000 [1.00, 1.00]
	benchmark	1.000 [1.00, 1.00]	0.000 [0.00, 0.00]	1.000 [1.00, 1.00]	1.000 [1.00, 1.00]
LLaMA 3 8B (Together)	objective	1.000 [1.00, 1.00]	0.000 [0.00, 0.00]	1.000 [1.00, 1.00]	1.000 [1.00, 1.00]
	method	1.000 [1.00, 1.00]	0.000 [0.00, 0.00]	1.000 [1.00, 1.00]	1.000 [1.00, 1.00]
	key_result	1.000 [1.00, 1.00]	0.000 [0.00, 0.00]	1.000 [1.00, 1.00]	1.000 [1.00, 1.00]
	model_or_system	1.000 [1.00, 1.00]	0.000 [0.00, 0.00]	1.000 [1.00, 1.00]	1.000 [1.00, 1.00]
	benchmark	1.000 [1.00, 1.00]	0.000 [0.00, 0.00]	1.000 [1.00, 1.00]	1.000 [1.00, 1.00]
Gemini 2.5 Pro (RAG)	objective	0.440 [0.22, 0.67]	0.170 [0.07, 0.27]	0.843 [0.74, 0.94]	0.976 [0.96, 0.99]
	method	0.270 [0.11, 0.47]	0.223 [0.13, 0.33]	0.777 [0.66, 0.87]	0.975 [0.96, 0.99]
	key_result	0.100 [0.04, 0.16]	0.244 [0.15, 0.34]	0.775 [0.69, 0.86]	0.969 [0.96, 0.98]
	model_or_system	0.720 [0.52, 0.92]	0.142 [0.03, 0.28]	0.832 [0.67, 0.96]	0.977 [0.95, 1.00]
	benchmark	0.470 [0.29, 0.64]	0.168 [0.08, 0.27]	0.816 [0.72, 0.90]	0.976 [0.96, 0.99]
Claude Sonnet 4.5 (RAG)	objective	0.040 [0.00, 0.09]	0.289 [0.22, 0.36]	0.720 [0.65, 0.79]	0.956 [0.94, 0.97]
	method	0.070 [0.00, 0.20]	0.296 [0.23, 0.36]	0.696 [0.61, 0.78]	0.959 [0.95, 0.97]
	key_result	0.110 [0.03, 0.23]	0.245 [0.17, 0.31]	0.785 [0.73, 0.84]	0.969 [0.96, 0.98]
	model_or_system	0.490 [0.23, 0.76]	0.244 [0.10, 0.39]	0.772 [0.63, 0.91]	0.972 [0.96, 0.99]
	benchmark	0.300 [0.18, 0.42]	0.255 [0.17, 0.34]	0.762 [0.66, 0.85]	0.967 [0.95, 0.98]
LLaMA 3 8B (RAG)	objective	1.000 [1.00, 1.00]	0.000 [0.00, 0.00]	1.000 [1.00, 1.00]	1.000 [1.00, 1.00]
	method	1.000 [1.00, 1.00]	0.000 [0.00, 0.00]	1.000 [1.00, 1.00]	1.000 [1.00, 1.00]
	key_result	1.000 [1.00, 1.00]	0.000 [0.00, 0.00]	1.000 [1.00, 1.00]	1.000 [1.00, 1.00]
	model_or_system	1.000 [1.00, 1.00]	0.000 [0.00, 0.00]	1.000 [1.00, 1.00]	1.000 [1.00, 1.00]
	benchmark	1.000 [1.00, 1.00]	0.000 [0.00, 0.00]	1.000 [1.00, 1.00]	1.000 [1.00, 1.00]
Mistral 7B (RAG)	objective	1.000 [1.00, 1.00]	0.000 [0.00, 0.00]	1.000 [1.00, 1.00]	1.000 [1.00, 1.00]
	method	1.000 [1.00, 1.00]	0.000 [0.00, 0.00]	1.000 [1.00, 1.00]	1.000 [1.00, 1.00]
	key_result	1.000 [1.00, 1.00]	0.000 [0.00, 0.00]	1.000 [1.00, 1.00]	1.000 [1.00, 1.00]
	model_or_system	1.000 [1.00, 1.00]	0.000 [0.00, 0.00]	1.000 [1.00, 1.00]	1.000 [1.00, 1.00]
	benchmark	1.000 [1.00, 1.00]	0.000 [0.00, 0.00]	1.000 [1.00, 1.00]	1.000 [1.00, 1.00]
Gemma 2 9B (RAG)	objective	1.000 [1.00, 1.00]	0.000 [0.00, 0.00]	1.000 [1.00, 1.00]	1.000 [1.00, 1.00]
	method	1.000 [1.00, 1.00]	0.000 [0.00, 0.00]	1.000 [1.00, 1.00]	1.000 [1.00, 1.00]
	key_result	1.000 [1.00, 1.00]	0.000 [0.00, 0.00]	1.000 [1.00, 1.00]	1.000 [1.00, 1.00]
	model_or_system	1.000 [1.00, 1.00]	0.000 [0.00, 0.00]	1.000 [1.00, 1.00]	1.000 [1.00, 1.00]
	benchmark	1.000 [1.00, 1.00]	0.000 [0.00, 0.00]	1.000 [1.00, 1.00]	1.000 [1.00, 1.00]
API-only aggregate (n=6 stacks where EMR<1)					
objective		0.532	0.133	0.876	0.981
method		0.401	0.180	0.822	0.974
key_result		0.431	0.147	0.865	0.980
model_or_system		0.749	0.128	0.852	0.980
benchmark		0.619	0.159	0.847	0.979
Conclusion-relevant avg		0.455	0.154	0.854	0.978
Metadata avg		0.684	0.144	0.850	0.979
Δ (metadata − conclusion)		0.229	-0.010	-0.005	0.001

Hypothesis verdict. A natural hypothesis is that BERTScore F1 would be *lower* on conclusion-relevant fields than on metadata fields. On the API-only aggregate (n=6 stacks), BERTScore F1 is 0.978 for conclusion-relevant fields and 0.979 for metadata fields ($\Delta = +0.0010$, paired benchmark vs. key_result Cohen's $d = -0.10$); the metric is therefore saturated on both groups and *cannot* discriminate between them, which is itself an important finding—BERTScore obscures rather than reveals the substantive divergence. EMR, by contrast, shows a sharp separation: 0.455 (conclusion-relevant) vs. 0.684 (metadata), Δ=+0.229, paired Cohen's d=+1.41. This validates the manuscript's claim that the divergence concentrates on the substantive payload (*objective*, *method*,

key_result) and supports adopting a multi-metric three-level framework rather than relying on aggregate BERTScore alone.